(1) Given an mxn matrix of 0's & 1's.

e.g.:

0	0	1	0	0	1
1	0	0	1	1	1
0	1	1	0	1	0
1	1	0	1	0	1
0	1	1	0	0	1

A robot have to go from anywhere in the first column to anywhere in the last column

- Robot may only go in →, ↗ or ↘ directions
- It may only step on 1's.

Find the number of paths using Dynamic Programming (DP).

(a) Set up a recurrence relation and briefly describe DP steps.

(b) Apply your DP algorithm to the above instance.

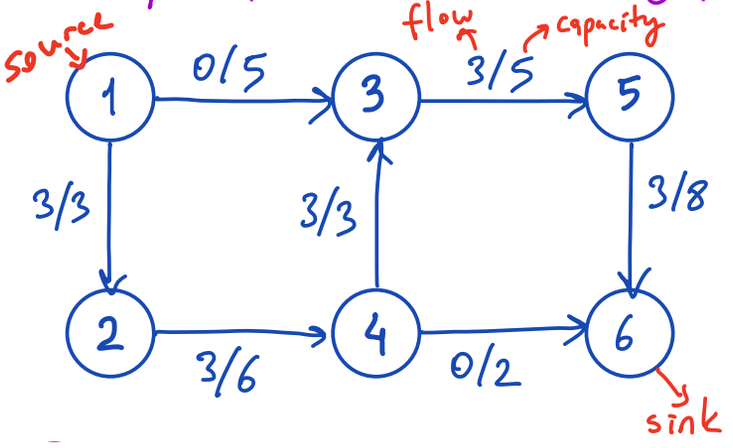
(c) What's the time complexity? (22pts)

(2) Consider a file F that consists of 4 chars.

char	frequency
A	40%
G	x%
C	y%
T	z%

x, y, z are nonzero integer values. What would be the maximum & minimum compression ratios when we apply Huffman Coding? Draw the Huffman trees and give the codewords for both cases. (22 pts)

(3) Ayse tells that this is an intermediate step of shortest augmenting path algorithm:

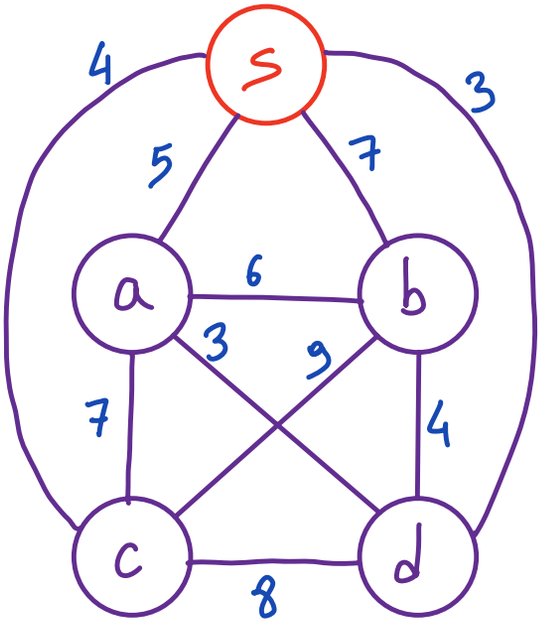


a) Do you think that Ayse has applied the algorithm correctly upto now? Why?

b) Regardless of your answer in (a) continue from this intermediate step, and apply shortest augmenting path algorithm to find the maximum flow. Show the labels and BFS queue in every step.

c) What is the minimum cut? (22pts)

(4) MAX-TSP-X1 problem is defined as follows:
Given a graph, each edge has a value (not cost).
Start from vertex s, visit ALL nodes EXCEPT ONE, go back to s. Find the tour with MAXIMUM value (not minimum).



Apply Branch-and-Bound algorithm to this instance. CLEARLY show your bound calculations in every step. Otherwise your answer will not be graded. (22 pts)

---

(5) Consider the MAX-TSP-X1 problem defined above. Answer the following:

a) What is the trivial lower bound of MAX-TSP-X1? Is this lower bound tight? Why?

b) Give a definition of decision version of MAX-TSP-X1, namely MAX-TSP-X1-Decide.

c) Show that MAX-TSP-X1-Decide is in NP.

d) Prove that MAX-TSP-X1-Decide is NP-Complete. (Use Problem Reduction.)

(22 pts)

Answers:

1 a) Define $P[i,j]$ as the number of paths to i^{th} row, j^{th} column.

Let's say $A[i,j]$ is the $m \times n$ matrix.

$P[i,1] = A[i,1]$, $P[0,j] = 0$, $P[n+1,j] = 0$ Base case

$P[i,j] = 0$ if $A[i,j] = 0$.

$P[i,j] = P[i-1,j-1] + P[i,j-1] + P[i+1,j-1]$ if $A[i,j] = 1$.

8 pts

In DP algorithm, calculate the 2nd column values, then the 3rd, until the last column, using the RR.

Number of paths $= \sum_i P[i,n]$ 4 pts

b)

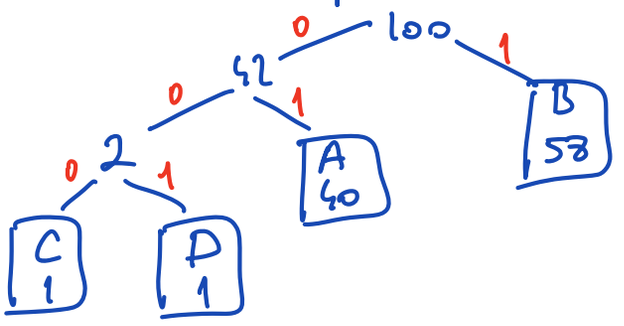
0	0	1	0	0	1		0	0	0	0	0	3
1	0	0	1	1	1		1	0	0	3	3	11
0	1	1	0	1	0	⇒	0	2	3	0	8	0
1	1	0	1	0	1		1	1	0	5	0	8
0	1	1	0	0	1		0	1	2	0	0	0

8 pts

paths $= 3 + 11 + 8 = 22$ //

c) Time complexity is $\Theta(n.m)$ 2 pts

(2) Maximum compression: A:40% B:58% C:1% D:1%

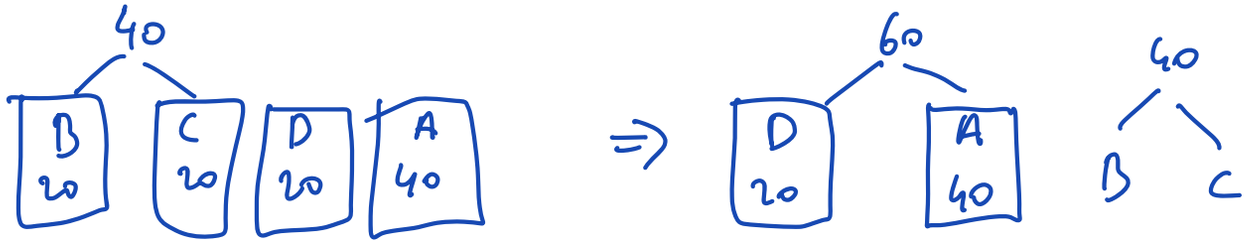


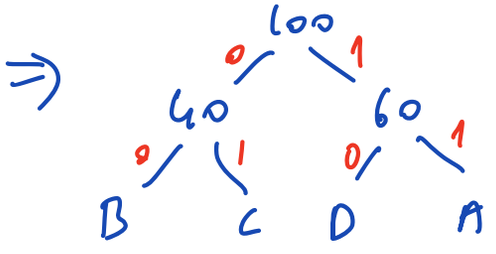
A:01
B:1
C:000
D:001

Avg codeword lngth = $0.58 \times 1 + 0.4 \times 2 + 0.01 \times 3 + 0.01 \times 3 = 1.44$

Compression rate = $\frac{2 - 1.44}{2} = \frac{0.56}{2} = 28\%$

Minimum compression: A:40% B:20% C:20% D:20%





A:11
B:00
C:01
D:10

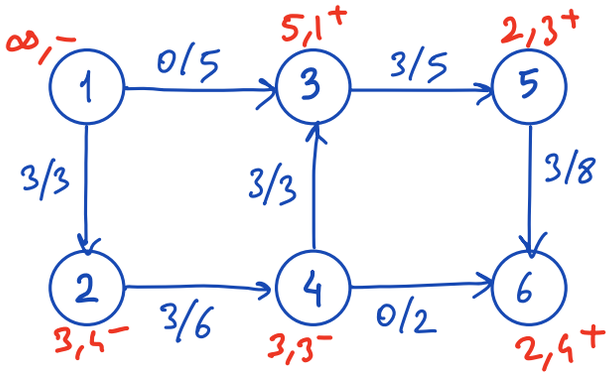
Avg codeword lngth = 2

Compression rate = 0%

(3) a) Ayse has not applied correctly. Because BFS finds shortest augmenting path, whereas Ayse found the augmenting path of $1 \to 2 \to 3 \to 5 \to 6$ which is obviously not a shortest path.

4 pts

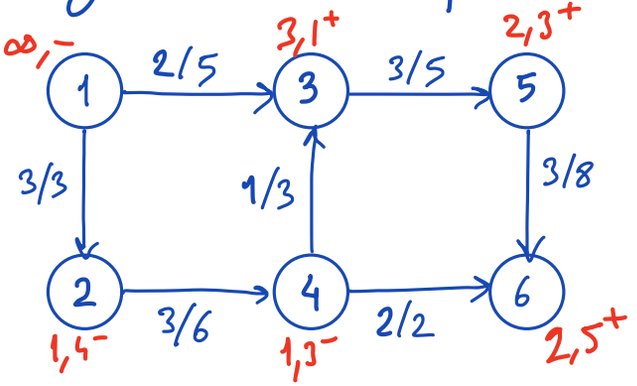
b)



BFS Queue: 1 3 4 5 2 6

Augmenting path $\Rightarrow 1 \to 3 \leftarrow 4 \to 6$

flow = 2

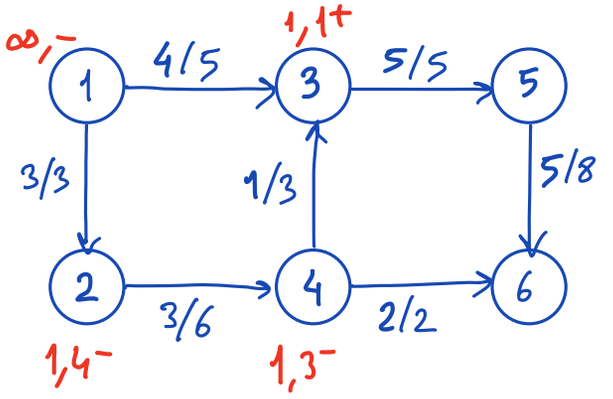


BFS Queue: 1 3 4 5 2 6

Aug. path $\Rightarrow 1 \to 3 \to 5 \to 6$

flow = 2

14 pts
queue: 3pts
labels: 4pts
aug. paths: 7pts

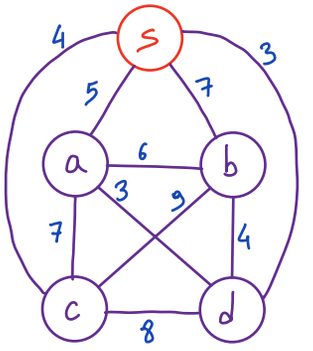


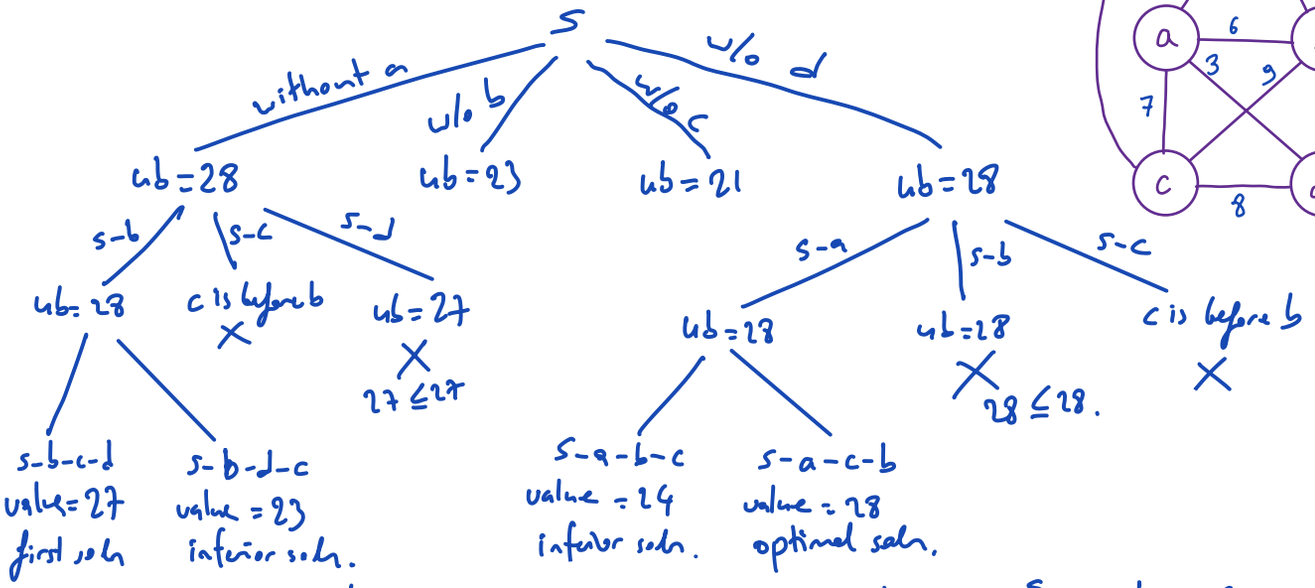
BFS Queue: 1 3 4 2

c) Minimum cut = $\{(3,5),(4,6)\}$ capacity of this cut = $5+2=7$ //

4 pts

④ Since this is a maximization problem, we have upper bounds instead of lower bounds.





w/o a: $\left\lfloor \frac{\overbrace{4+7}^{s} + \overbrace{7+9}^{b} + \overbrace{8+9}^{c} + \overbrace{8+4}^{d}}{2} \right\rfloor = 28$

w/o b: $\left\lfloor \frac{\overbrace{5+4}^{s} + \overbrace{7+5}^{a} + \overbrace{8+7}^{c} + \overbrace{8+3}^{d}}{2} \right\rfloor = 23$

w/o c: $\left\lfloor \frac{\overbrace{7+5}^{s} + \overbrace{6+5}^{a} + \overbrace{7+6}^{b} + \overbrace{4+3}^{d}}{2} \right\rfloor = 21$

w/o d: $\left\lfloor \frac{\overbrace{7+5}^{s} + \overbrace{7+6}^{a} + \overbrace{9+7}^{b} + \overbrace{9+7}^{c}}{2} \right\rfloor = 28$

w/o a:

s-b: $\left\lfloor \frac{\overbrace{7+4}^{s} + \overbrace{7+9}^{b} + \overbrace{8+9}^{c} + \overbrace{8+4}^{d}}{2} \right\rfloor = 28$

s-d: $\left\lfloor \frac{\overbrace{7+3}^{s} + \overbrace{7+9}^{b} + \overbrace{8+9}^{c} + \overbrace{3+8}^{d}}{2} \right\rfloor = 27$

w/o d:

s-a: $\left\lfloor \frac{\overbrace{7+5}^{s} + \overbrace{7+5}^{a} + \overbrace{9+7}^{b} + \overbrace{9+7}^{c}}{2} \right\rfloor = 28$

s-b: $\left\lfloor \frac{\overbrace{7+5}^{s} + \overbrace{7+6}^{a} + \overbrace{7+9}^{b} + \overbrace{7+9}^{c}}{2} \right\rfloor = 28$

(5) a) Trivial LB of MAX-TSP-X1 is $\Omega(n)$ since we have to process all the inputs in order to find a solution. This LB is not tight, since there is no linear time algorithm to solve the problem. 2+2=4 pts

b) MAX-TSP-X1-Decide can be defined as follows: Starting from node s, find a Hamiltonian cycle that visits all the nodes except one and have a total value greater than K. 4pts

c) Given a potential solution, verification can be done in linear time. We just check whether n-1 nodes are visited exactly once, and sum of the values of the edges are greater than K. 4 pts

d) It is easy to reduce decision version of ordinary TSP problem to MAX-TSP-X1-Decide. First, we revise all the edge weights (w_{ij}) with MAX-w_{ij} where MAX is the maximum weight. Then we add an arbitrary node x, s.t. any edge between x and other nodes has a weight of zero. Then we apply MAX-TSP-X1 algorithm. If the threshold for the ordinary TSP is K, and number of nodes (except x) is n, the threshold for MAX-TSP-X1-Decide is set to n.MAX-K.

Hence, MAX-TSP-X1-Decide is NP (proven in c) and an NP-Complete problem (TSP) can be reduced to it in linear time. We conclude that MAX-TSP-X1-Decide is NP-Complete. 10 pts

